

Legacy-to-AWS Migration Architecture Package

Repository: COG-GTM/Springboot-BankApp (branch DevOps)

Status: For architecture review. Docs-only; no application code changed.

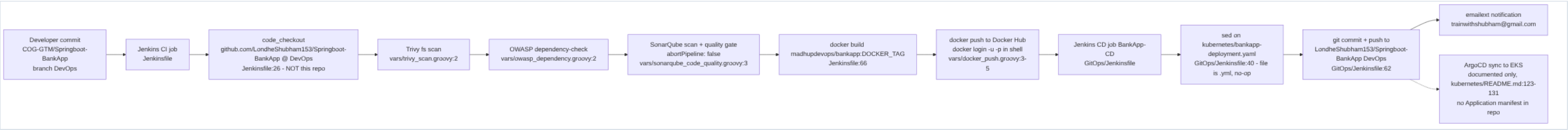
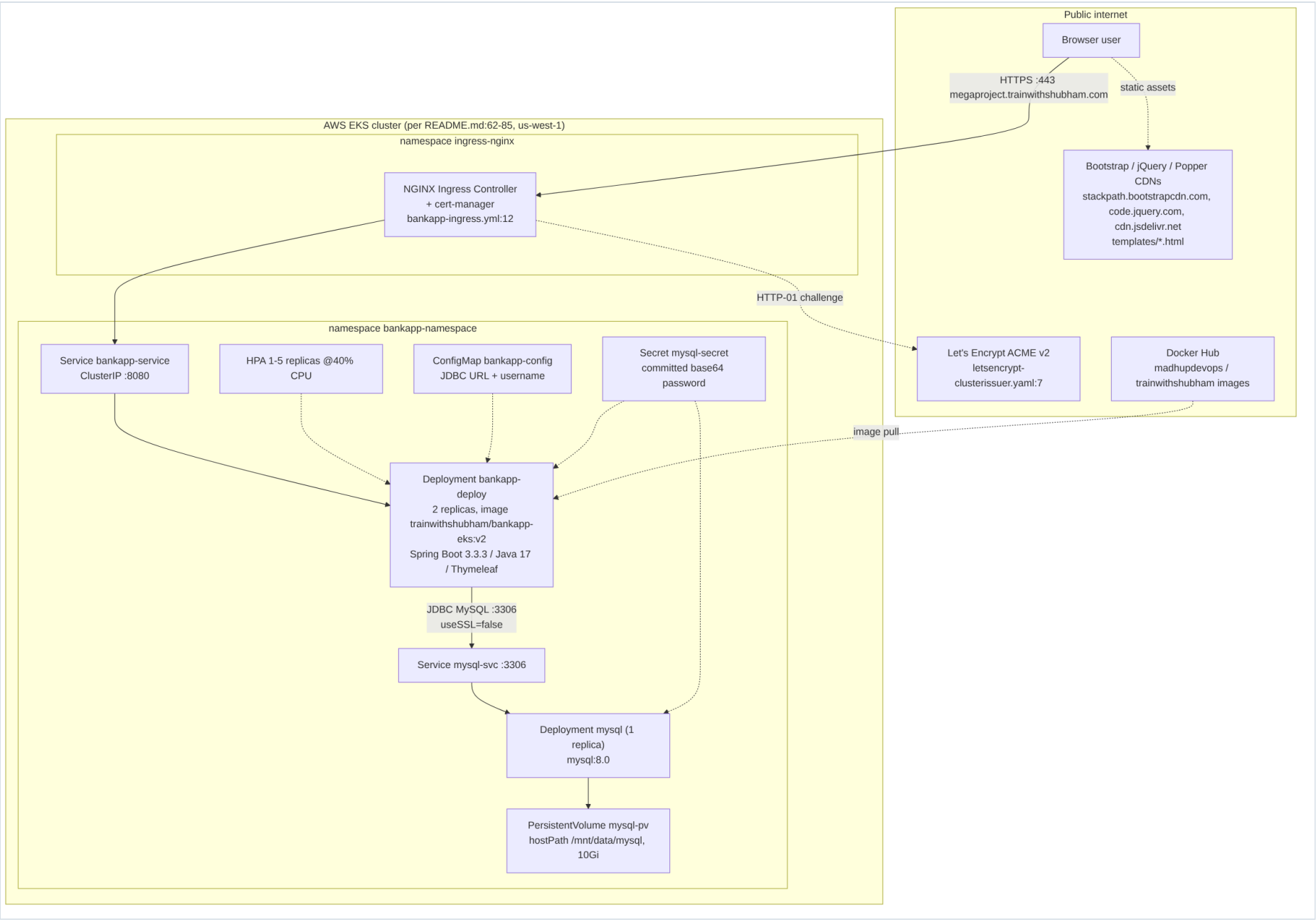
Assumed inputs. No approved ("blessed") AWS service list and no platform migration guardrails were supplied. The service list and the ten guardrails in the Target State document are explicitly labelled assumptions made so design could start; they are not customer policy. Confirming or replacing them is open question Q1 and Q2.

Contents

- [Current State](#) — docs/architecture/current-state.md
- [Target State](#) — docs/architecture/target-state.md
- [Open Questions](#) — docs/architecture/open-questions.md
- [Migration Plan](#) — docs/architecture/migration-plan.md

Current State — COG-GTM/Springboot-BankApp

Scope: everything below is derived from the files on branch `DevOps` of this repository. Every statement carries a `file:line` citation. Statements that were not read directly out of a file are prefixed **Inference:**.



1. Application runtime

Fact	Evidence
Single Spring Boot service, artifact <code>com.example:bankapp:0.0.1-SNAPSHOT</code>	<code>pom.xml:11-13</code>

Fact	Evidence
Spring Boot parent 3.3.3	<code>pom.xml:8</code>
<code>java.version</code> property 17	<code>pom.xml:30</code>
<code>maven-compiler-plugin</code> pinned to <code>source / target 1.8</code> , contradicting the Java 17 property	<code>pom.xml:78-86</code>
Starters in use: <code>data-jpa</code> , <code>security</code> , <code>thymeleaf</code> , <code>web</code> , <code>thymeleaf-extras-springsecurity6</code>	<code>pom.xml:33-52</code>
JDBC driver <code>mysql:mysql-connector-java:8.0.33</code> (the retired <code>mysql</code> coordinate, not <code>com.mysql:mysql-connector-j</code>)	<code>pom.xml:54-59</code>
Test dependencies: <code>spring-boot-starter-test</code> , <code>spring-security-test</code>	<code>pom.xml:60-69</code>
Entry point <code>BankappApplication</code> — a bare <code>@SpringBootApplication</code> with no additional configuration	<code>src/main/java/com/example/bankapp/BankappApplication.java:6-11</code>
Server-side rendered UI (Thymeleaf), four templates: <code>login</code> , <code>register</code> , <code>dashboard</code> , <code>transactions</code>	<code>src/main/resources/templates/</code>
HTTP session-based form login, session invalidated on logout	<code>src/main/java/com/example/bankapp/config/SecurityConfig.java:35-47</code>
Passwords hashed with BCrypt	<code>SecurityConfig.java:22-25</code> , <code>AccountService.java:45</code>
CSRF protection disabled	<code>SecurityConfig.java:30</code>
Only <code>/register</code> is anonymous; everything else requires authentication	<code>SecurityConfig.java:31-34</code>
A single hardcoded authority <code>"USER"</code> is granted to every account — no roles, no segregation of duties	<code>AccountService.java:99-101</code>

Endpoints (complete list)

Method	Path	Handler	Evidence
GET	<code>/dashboard</code>	<code>BankController.dashboard</code>	<code>BankController.java:21-27</code>
GET	<code>/register</code>	<code>BankController.showRegistrationForm</code>	<code>BankController.java:29-32</code>
POST	<code>/register</code>	<code>BankController.registerAccount</code>	<code>BankController.java:34-43</code>
GET	<code>/login</code>	<code>BankController.login</code>	<code>BankController.java:45-48</code>
POST	<code>/login</code>	Spring Security form login processing URL	<code>SecurityConfig.java:37</code>
POST	<code>/deposit</code>	<code>BankController.deposit</code>	<code>BankController.java:50-56</code>
POST	<code>/withdraw</code>	<code>BankController.withdraw</code>	<code>BankController.java:58-72</code>
GET	<code>/transactions</code>	<code>BankController.transactionHistory</code>	<code>BankController.java:74-80</code>
POST	<code>/transfer</code>	<code>BankController.transferAmount</code>	<code>BankController.java:82-96</code>
POST	<code>/logout</code>	Spring Security logout matcher	<code>SecurityConfig.java:41-47</code>

There are **no** REST/JSON endpoints: zero occurrences of `@RestController` in `src/`. The service is HTML-only, which means the migration has no API contract to preserve, only session behaviour.

2. Datastore and data access

Fact	Evidence
MySQL is the only datastore; JDBC URL <code>jdbc:mysql://localhost:3306/bankappdb?useSSL=false&serverTimezone=UTC</code> in the packaged properties	<code>src/main/resources/application.properties:3</code>
Username <code>root</code> and a plaintext password are committed to the repository	<code>application.properties:4-5</code>
Hibernate dialect pinned to <code>MySQL8Dialect</code>	<code>application.properties:10</code>
<code>spring.jpa.hibernate.ddl-auto=update</code> — the schema is created and mutated by Hibernate at runtime	<code>application.properties:9</code>
<code>spring.jpa.show-sql=true</code> — SQL logged on every statement in all environments	<code>application.properties:11</code>
Two JPA entities: <code>Account</code> (id, username, password, balance, transactions) and <code>Transaction</code> (id, amount, type, timestamp, account)	<code>model/Account.java:11-25</code> , <code>model/Transaction.java:7-19</code>
Two Spring Data repositories, derived queries only: <code>findByUsername</code> , <code>findById</code>	<code>repository/AccountRepository.java:8-10</code> , <code>repository/TransactionRepository.java:8-10</code>
The only SQL artefact in the repo is a one-line database create script	<code>src/main/resources/static/mysql/SQLScript.txt:1</code>

Zero-count negatives (load-bearing for the migration)

Searched across `src/**/*.java`, `pom.xml` and `*.properties` :

Pattern	Occurrences	Why it matters
@Query / nativeQuery	0	No hand-written JPQL or native SQL to port to PostgreSQL
JdbcTemplate , EntityManager	0	No lower-level SQL access paths
Stored procedures (CALL , @Procedure)	0	No database-side logic to migrate
@Scheduled	0	No batch or cron behaviour inside the application
@Transactional	0	No transaction boundary is declared anywhere, including the multi-write transfer
RestTemplate , WebClient	0	No outbound HTTP calls from application code
Kafka / JMS / RabbitMQ	0	No messaging
new File , FileInputStream , java.nio	0	No file I/O; nothing stateful on the container filesystem
redis	0	Sessions are in-memory per JVM (Inference: default Spring Session behaviour, no session store is configured)
Flyway / Liquibase	0	No migration tooling — schema evolution relies entirely on <code>ddl-auto=update</code>
spring-boot-starter-actuator	0	No actuator dependency, yet health probes target <code>/actuator/health</code> (see §6)

3. Outbound integrations

Integration	Direction	Evidence
MySQL over JDBC, TLS explicitly disabled (<code>useSSL=false</code>)	app → DB	<code>application.properties:3</code> , <code>kubernetes/configmap.yaml:8</code> , <code>docker-compose.yml:25</code>
Bootstrap 4.5.2 CSS/JS from <code>stackpath.bootstrapcdn.com</code>	browser → public CDN	<code>templates/login.html:5</code> , <code>templates/register.html:5</code> , <code>templates/dashboard.html:5</code> , <code>templates/dashboard.html:196</code> , <code>templates/transactions.html:5</code>
jQuery 3.5.1 from <code>code.jquery.com</code>	browser → public CDN	<code>templates/dashboard.html:194</code>
Popper 2.5.3 from <code>cdn.jsdelivr.net</code>	browser → public CDN	<code>templates/dashboard.html:195</code>
Let's Encrypt ACME v2 (HTTP-01 solver) for TLS certificates	cluster → internet	<code>kubernetes/letsencrypt-clusterissuer.yaml:6-14</code>
Docker Hub as image registry (push and pull)	CI → registry, cluster → registry	<code>Jenkinsfile:74</code> , <code>vars/docker_push.groovy:5</code> , <code>kubernetes/bankapp-deployment.yml:20</code>
SMTP notification from the CD job to <code>trainwithshubham@gmail.com</code>	Jenkins → mail relay	<code>GitOps/Jenkinsfile:72-91</code>
GitHub push from the CD job	Jenkins → GitHub	<code>GitOps/Jenkinsfile:62</code>

4. Hosting model

Fact	Evidence
Target platform documented as AWS EKS in <code>us-west-1</code> , cluster <code>bankapp</code> , <code>t2.medium</code> nodes created with <code>eksctl</code>	<code>README.md:27</code> , <code>README.md:62-85</code>
Application Deployment: 2 replicas, container port 8080, requests 250m/512Mi, limits 500m/1Gi	<code>kubernetes/bankapp-deployment.yml:9</code> , <code>:22</code> , <code>:56-62</code>
Image referenced by the Deployment: <code>trainwithshubham/bankapp-eks:v2</code>	<code>kubernetes/bankapp-deployment.yml:20</code>
Readiness and liveness probes are commented out in the Kubernetes Deployment	<code>kubernetes/bankapp-deployment.yml:44-55</code>
HPA: 1–5 replicas at 40% average CPU (minReplicas 1 undercuts the Deployment's <code>replicas: 2</code>)	<code>kubernetes/bankapp-hpa.yml:11-19</code>
Service <code>bankapp-service</code> , ClusterIP, 8080 → 8080	<code>kubernetes/bankapp-service.yaml:8-14</code>
Ingress: NGINX class, forced SSL redirect, cert-manager issuer <code>letsencrypt-prod</code> , host <code>megaproject.trainwithshubham.com</code>	<code>kubernetes/bankapp-ingress.yml:6-18</code>
MySQL runs inside the cluster as a 1-replica Deployment of <code>mysql:8.0</code>	<code>kubernetes/mysql-deployment.yml:9</code> , <code>:20</code>
MySQL storage is a <code>hostPath</code> <code>PersistentVolume</code> at <code>/mnt/data/mysql</code> , 10Gi, <code>storageClassName: standard</code>	<code>kubernetes/persistent-volume.yaml:7-16</code> , <code>kubernetes/persistent-volume-claim.yaml:6-12</code>
DB connection details injected from a ConfigMap; password from Secret <code>mysql-secret</code>	<code>kubernetes/bankapp-deployment.yml:23-43</code> , <code>kubernetes/configmap.yaml:6-9</code>
An alternative Helm chart exists and disagrees with the raw manifests (see §6)	<code>helm/bankapp/**</code>
Local/dev path is Docker Compose: <code>mysql:latest</code> + app image <code>`\${DUSER}/\${IMAGE}</code>	<code>docker-compose.yml:3-34</code>
<code>.env</code> resolves that image to <code>joakim077/springboot-application</code>	<code>.env:1-2</code>

Fact	Evidence
Container image build: Maven 3.8.3/OpenJDK 17 builder stage, <code>openjdk:17-alpine</code> runtime, tests skipped, runs as root (no <code>USER</code> directive)	Dockerfile:6 , :21 , :28 , :31-37

5. Delivery path to production

Step	Evidence
CI is Jenkins declarative pipeline, <code>agent any</code> , with a <code>Shared</code> global library	Jenkinsfile:1-3
CI checks out <code>https://github.com/LondheShubham153/Springboot-BankApp.git</code> branch <code>DevOps</code> — an upstream third-party repository, not this one	Jenkinsfile:26
Scan stages: Trivy filesystem, OWASP dependency-check, SonarQube analysis, SonarQube quality gate	Jenkinsfile:31-61
The quality gate is advisory: <code>waitForQualityGate abortPipeline: false</code>	vars/sonarqube_code_quality.groovy:3
Trivy runs <code>trivy fs .</code> with no severity threshold and no <code>--exit-code</code> , so it can never fail the build	vars/trivy_scan.groovy:2
Image build/push to Docker Hub namespace <code>madhupdevops</code> , tag from the <code>DOCKER_TAG</code> build parameter (default empty)	Jenkinsfile:10 , :66 , :74
Docker Hub credentials passed on the command line (<code>docker login -u ... -p ...</code>), which exposes them in the process list and build log	vars/docker_push.groovy:2-5
On success CI triggers the downstream job <code>BankApp-CD</code>	Jenkinsfile:79-85
CD checks out the same upstream repository	GitOps/Jenkinsfile:21
CD rewrites the image tag with <code>sed</code> on <code>kubernetes/bankapp-deployment.yaml</code>	GitOps/Jenkinsfile:40
CD commits and pushes to the upstream repository's <code>DevOps</code> branch	GitOps/Jenkinsfile:50-63
ArgoCD is the documented sync mechanism, created imperatively via CLI; no <code>Argo Application</code> manifest is committed	kubernetes/README.md:116-133 , README.md:266-275
Only automated test in the repository is <code>contextLoads()</code>	src/test/java/com/example/bankapp/BankappApplicationTests.java:9-11
The Dockerfile builds with <code>-DskipTests=true</code> , so even that test does not run in the image build	Dockerfile:21
No GitHub Actions workflows exist (<code>.github/</code> is absent from the tree)	repository file listing

6. Contradictions between artefacts

These are the highest-value findings: the artefacts do not describe one system.

#	Contradiction	Evidence
C 1	The pipeline does not build this repository. Both Jenkins jobs check out <code>LondheShubham153/Springboot-BankApp</code> . Any change committed here is never built, scanned or deployed.	Jenkinsfile:26 , GitOps/Jenkinsfile:21
C 2	The CD <code>sed</code> targets a filename that does not exist. It edits <code>bankapp-deployment.yaml</code> ; the file in the repo is <code>bankapp-deployment.yml</code> . The tag update is a silent no-op and the commit step then commits nothing.	GitOps/Jenkinsfile:40 vs kubernetes/bankapp-deployment.yml
C 3	Four different image references for one application. CI pushes <code>madhupdevops/bankapp:\$DOCKER_TAG</code> ; the Kubernetes Deployment runs <code>trainwithshubham/bankapp-eks:v2</code> ; the Helm chart runs <code>trainwithshubham/springboot-bankapp:latest</code> ; Compose runs <code>joakim077/springboot-application</code> .	Jenkinsfile:66 , kubernetes/bankapp-deployment.yml:20 , helm/bankapp/values.yaml:26 , .env:1-2
C 4	Health probes point at an endpoint that cannot exist. The Helm Deployment and the Compose healthcheck probe <code>/actuator/health</code> , but <code>spring-boot-starter-actuator</code> is not a dependency, so that path returns 404 (and, given <code>anyRequest().authenticated()</code> , a redirect to <code>/login</code> for unauthenticated probes). The raw Kubernetes Deployment sidesteps this by commenting the probes out entirely.	helm/bankapp/templates/deployment.yml:43-54 , docker-compose.yml:36 , pom.xml:32-70 , SecurityConfig.java:33 , kubernetes/bankapp-deployment.yml:44-55
C 5	Two incompatible deployment descriptions. Raw manifests: MySQL Deployment + <code>mysql-svc</code> + hostPath PV, app <code>Service</code> ClusterIP behind an NGINX Ingress on <code>megaproject.trainwithshubham.com</code> . Helm chart: MySQL <code>StatefulSet</code> + headless service + volumeClaimTemplate, app <code>Service</code> type <code>NodePort</code> 30080, Ingress host <code>bankapp.local</code> , replicas 1, different CPU/memory (80m–800m / 150Mi–700Mi), plus a VPA in <code>Auto</code> mode alongside a CPU HPA.	kubernetes/*.yaml vs helm/bankapp/templates/*.yaml , helm/bankapp/values.yaml:17-45 , helm/bankapp/templates/vpa.yaml:12
C 6	Build targets Java 8 while the framework requires 17. <code>java.version</code> is 17 but <code>maven-compiler-plugin</code> is explicitly configured <code>source / target</code> 1.8. Inference: the explicit plugin configuration overrides the Spring Boot parent's <code>maven.compiler.release</code> , so the declared bytecode target contradicts Spring Boot 3.3.3's Java 17 baseline.	pom.xml:30 , pom.xml:78-86
C 7	README claims a GitOps pipeline the repo does not contain. ArgoCD is described as the deployment mechanism, but no <code>Application</code> CR is committed anywhere; the app is created by an interactive <code>argocd app create</code> command with placeholder arguments.	README.md:11-16 , kubernetes/README.md:121-131
C 8	HPA <code>minReplicas: 1</code> fights the Deployment's <code>replicas: 2</code>, so the documented "keep replicas >= 2 for high availability" intent is discarded as soon as the HPA takes ownership.	kubernetes/bankapp-deployment.yml:9 , kubernetes/bankapp-hpa.yml:11

7. Correctness and security facts carried into the plan

Reported here as current-state facts, not fixed in this docs-only change. They gate production cutover (see `migration-plan.md`, WS6).

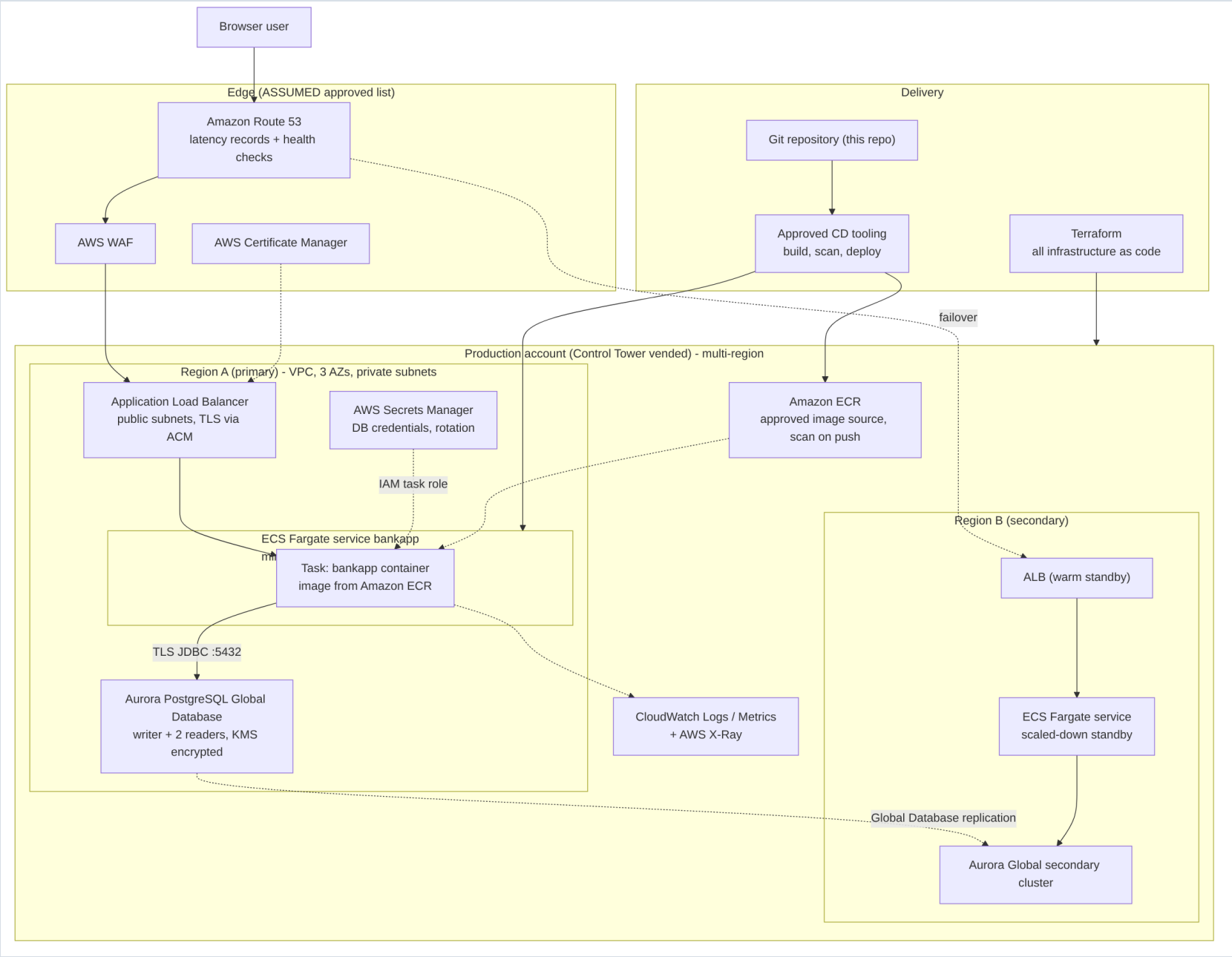
#	Fact	Evidence
S1	Database credentials are committed to the repository in four places (application properties, Compose file, Kubernetes Secret as base64, Helm values in plaintext). Base64 is encoding, not protection. Values are not reproduced here.	<code>application.properties:4-5</code> , <code>docker-compose.yml:7</code> , <code>docker-compose.yml:26</code> , <code>kubernetes/secrets.yml:8-9</code> , <code>helm/bankapp/values.yml:52-53</code>
S2	<code>transferAmount</code> performs four writes (two balance saves, two transaction rows) with no transaction boundary; a mid-sequence failure leaves the ledger inconsistent.	<code>AccountService.java:103-135</code> , zero <code>@Transactional</code> in <code>src/</code>
S3	No amount validation anywhere: <code>deposit</code> , <code>withdraw</code> and <code>transferAmount</code> accept any <code>BigDecimal</code> , including negatives, and <code>BankController</code> passes the request parameter straight through.	<code>AccountService.java:51-135</code> , <code>BankController.java:51</code> , <code>:59</code> , <code>:83</code>
S4	CSRF protection is disabled on a session-cookie application with state-changing POST endpoints.	<code>SecurityConfig.java:30</code>
S5	Every user gets the same <code>"USER"</code> authority; there is no authorisation model, no maker-checker, no limits.	<code>AccountService.java:99-101</code>
S6	Schema is mutated at runtime by Hibernate (<code>ddl-auto=update</code>), so the database structure is not a controlled artefact.	<code>application.properties:9</code>
S7	No audit logging of financial events; the only logging configured is <code>show-sql</code> .	<code>application.properties:11</code> , no logging statements in <code>src/main/java/**</code>
S8	Database traffic is unencrypted (<code>useSSL=false</code>) in every environment definition.	<code>application.properties:3</code> , <code>kubernetes/configmap.yml:8</code> , <code>docker-compose.yml:25</code>
S9	The container runs as root: the Dockerfile declares no <code>USER</code> .	<code>Dockerfile:28-37</code>
S10	Test coverage is effectively zero (one <code>contextLoads()</code>), so there is no regression net for an engine or platform change.	<code>BankappApplicationTests.java:9-11</code>

8. Stateful surface (what actually has to be migrated)

Inference: derived from the facts above rather than from a single file.

- One MySQL 8 database, two tables (`account` , `transaction`) generated by Hibernate from `Account.java` and `Transaction.java` . No other persistent state exists: no file I/O, no cache, no queue, no object storage.
- HTTP session state lives in each JVM's memory. With two replicas and no sticky-session annotation on the Ingress (`kubernetes/bankapp-ingress.yml:6-10` sets only `rewrite-target`, body size and `ssl-redirect`), a user's requests can land on a replica that does not hold their session.
- Production data volumes, transaction rates, RTO/RPO and whether this repository is what actually runs in production cannot be determined from the code — see `open-questions.md` .

Target State — AWS



0. Inputs that were not supplied — read this first

ASSUMED, NOT CUSTOMER-SUPPLIED. No approved ("blessed") AWS service list and no platform migration guardrails were provided for this engagement. Everything in §1 and §2 below is an assumption made so that design work could start. It is not the customer's policy and must not be quoted as such. Confirming or replacing both is open question **Q1** and **Q2** in `open-questions.md`, addressed to Platform Engineering, and both are entry criteria for WS0 in `migration-plan.md`.

Environment assumptions, also supplied as assumptions rather than confirmed requirements:

Assumption	Value
Dev resilience posture	Multi-AZ, single region
Production resilience posture	Multi-region (active/warm-standby)
Target RDBMS	Amazon Aurora PostgreSQL
Target compute	Amazon ECS on Fargate (containers-first)

Assumption	Value
IaC	Terraform
CD	The organisation's approved pipeline tooling (specific product unconfirmed — Q14)

1. ASSUMED approved service list

Only these services are used in the target design. Anything the design needed that is not on this list is tagged `OFF-LIST?` in §3 with an in-list alternative.

Domain	Services
Accounts and governance	AWS Organizations, AWS Control Tower, AWS IAM, AWS Config, AWS CloudTrail
Network	Amazon VPC, AWS Transit Gateway, Amazon Route 53, AWS WAF, Elastic Load Balancing (ALB)
Compute	Amazon ECS on AWS Fargate
Containers and artefacts	Amazon ECR
Data	Amazon Aurora PostgreSQL (incl. Global Database), Amazon S3, AWS Database Migration Service (DMS)
Security	AWS KMS, AWS Secrets Manager, AWS Systems Manager Parameter Store, AWS Certificate Manager, Amazon GuardDuty, AWS Security Hub
Observability	Amazon CloudWatch (Logs, Metrics, Alarms), AWS X-Ray
Integration	Amazon EventBridge, Amazon SNS

2. Platform migration guardrails in force

ASSUMED DEFAULTS. The customer supplied no guardrails; this is the playbook's default set, reproduced so that every design choice can be assessed against something explicit. Replace it with the customer's own once Q2 is answered.

#	Guardrail
G1	Approved-service allowlist — only services on the approved list may appear in the target architecture; the allowlist is assumed to be enforced organisation-wide by service control policies (SCPs).
G2	Containers first — the approved managed container platform is the preferred compute target; VM-based lift-and-shift requires explicit justification.
G3	Standard target RDBMS — relational workloads target the organisation's standard engine (PostgreSQL); staying on a non-standard engine is an exception that must be justified and flagged.
G4	Account vending and environment promotion — workloads are onboarded through the standard account-vending process (AWS Control Tower) with separate dev, QA and prod accounts; a dev account must be running before QA is granted; changes promote dev → QA → prod.
G5	Everything as code — all target infrastructure is defined in Terraform; no console-built resources.
G6	Approved delivery toolchain — CI/CD runs on the organisation's approved pipeline tooling; migrating off legacy pipelines is part of the plan.
G7	Production entry criteria — prod deployment requires multi-region, load balancing and a demonstrated DR/failover test before approval.
G8	Security baseline — private networking by default, encryption in transit and at rest, least-privilege IAM roles with no long-lived static credentials, secrets in the approved secrets manager.
G9	Approved artefact sources — container images and dependencies only from approved registries/repositories.
G10	Tagging and observability standards — mandatory tags (owner, cost centre, environment, data classification) and logs/metrics to the central observability platform.

3. Component-by-component target

Effort is engineering effort for one engineer, expressed in Devin sessions (one session ≈ one to two human-weeks of equivalent work).

#	Current component	Evidence	Target AWS service	Rationale	Risk	Effort
1	Spring Boot 3.3.3 web app on EKS, 2 replicas	kubernetes/bankapp-deployment.yml:9-20	Amazon ECS on Fargate , min 3 tasks spread across 3 AZs	G2 containers-first; the app is already a single stateless container with no daemonset/operator/CRD needs, so Fargate removes node-group ownership (README.md : 74 - 85) without a repackaging cost	L	1 session
2	Container image openjdk:17-alpine , root user, tests skipped	Dockerfile:21-37	Rebuilt image on an approved base image from ECR , non-root user, tests run in the build	G9 approved artefact sources; G8 least privilege. openjdk:17-alpine is an unmaintained Docker Hub tag	M	0.5 session

#	Current component	Evidence	Target AWS service	Rationale	Risk	Effort
3	Docker Hub registry (madhupdevops , trainwithshubham , joakim077)	Jenkinsfile:74 , .env:1-2	Amazon ECR with scan-on-push and immutable tags	G9; also resolves C3 — one image identity replaces four	L	0.5 session
4	In-cluster MySQL 8, 1 replica, hostPath PV	kubernetes/mysql-deployment.yml:9-41 , kubernetes/persistent-volume.yml:14-15	Aurora PostgreSQL (Global Database in prod), Multi-AZ, KMS-encrypted, automated backups	G3 standard engine; a single-replica hostPath database is not survivable — the data is pinned to one node's local disk	H	3 sessions (see WS3)
5	Hibernate MySQL8Dialect , ddl-auto=update	application.properties:9-10	PostgreSQLDialect , ddl-auto=validate , schema owned by a migration tool	Engine change plus G-compliant change control; zero native SQL and zero stored procedures (see current-state §2 negatives) is what keeps this from being a rewrite	M	included in row 4
6	Schema evolution by Hibernate at runtime	application.properties:9	Flyway or Liquibase running as a pipeline step (application-level tooling, not an AWS service)	Makes the schema a reviewable artefact; prerequisite for promoting dev → QA → prod under G4	M	0.5 session
7	NGINX Ingress Controller + cert-manager + Let's Encrypt	kubernetes/bankapp-ingress.yml:6-18 , kubernetes/letsencrypt-clusterissuer.yml:6-14	ALB + AWS Certificate Manager + AWS WAF	G7 load balancing; ACM removes the outbound ACME dependency and the renewal failure mode	L	0.5 session
8	Host megaproject.trainwithshubham.com	kubernetes/bankapp-ingress.yml:15-18	Route 53 record in the corporate zone, latency/failover routing across the two regions	Third-party demo domain cannot go to production; failover routing is the mechanism for G7 multi-region	L	0.25 session
9	Committed DB credentials (4 locations)	current-state §7 S1	AWS Secrets Manager, injected into the task definition by ARN, rotation enabled	G8 no secrets in code. Rotation and revocation of the exposed values is WS1 work, not a rename	H	1 session
10	Non-secret config in ConfigMap	kubernetes/configmap.yml:6-9	SSM Parameter Store parameters referenced by the task definition	G5/G8; keeps environment differences out of the image	L	0.25 session
11	HPA 1–5 @40% CPU (plus a conflicting VPA in the Helm chart)	kubernetes/bankapp-hpa.yml:11-19 , helm/bankapp/templates/vpa.yml:12	ECS Service Auto Scaling target-tracking on CPU, min 3 / max 10	Single scaling mechanism; min 3 keeps one task per AZ, resolving C8	L	0.25 session
12	Jenkins CI + Jenkins GitOps CD + ArgoCD (documented only)	Jenkinsfile , GitOps/Jenkinsfile , kubernetes/README.md:121-131	Approved CD tooling driving build → ECR → ECS deploy, with Trivy/SCA/SAST gates that actually fail the build	G6. Also fixes C1 (pipeline builds the wrong repository) and C2 (sed on a non-existent filename)	H	2 sessions
13	Raw Kubernetes manifests + a divergent Helm chart	kubernetes/** , helm/**	Terraform modules (VPC, ALB, ECS, Aurora, IAM, observability); both existing descriptor sets retired	G5 everything as code; C5 disappears because there is one description of the system	M	2 sessions
14	No probes (K8s) / probes on a non-existent /actuator/health (Helm, Compose)	kubernetes/bankapp-deployment.yml:44-55 , helm/bankapp/templates/deployment.yml:43-54	Add spring-boot-starter-actuator , expose /actuator/health unauthenticated, wire it to the ALB target-group health check and the ECS container health check	Without a real health endpoint the ALB cannot deregister a broken task; this is a prerequisite for G7	M	0.25 session
15	In-JVM HTTP sessions across 2 replicas, no sticky sessions	current-state §8	ALB sticky sessions (application cookie) as the interim, stateless auth as the target	Preserves behaviour on day one without adding a session store; see the OFF-LIST? row below (Q13)	M	0.5 session
16	show-sql=true , no application logging, no metrics	application.properties:11	CloudWatch Logs (structured JSON via awslogs), CloudWatch metrics and alarms, X-Ray tracing	G10; also the only way to evidence the audit-logging gap S7 being closed	L	0.5 session
17	Bootstrap/jQuery/Popper from public CDNs	templates/dashboard.html:194-196	Vendored into the image from the approved internal artefact repository, served by the app	G9 approved sources; removes three public-internet dependencies from a page that renders account balances	L	0.25 session
18	Jenkins emailext build notification	GitOps/Jenkinsfile:72-91	Amazon SNS topic subscribed by the delivery team	G1 keeps notification on an in-list service	L	0.1 session
19	Manual cluster creation with eksctl by a root user	README.md:20-23 , :62-85	Control Tower-vended dev / QA / prod accounts, Terraform-provisioned	G4; sudo su plus static IAM access keys is the opposite of G8	M	1 session (WS0)
20	MySQL data on a node's local disk	kubernetes/persistent-volume.yml:14-15	DMS for the migration itself, S3 for dumps/validation artefacts, Aurora automated backups + PITR thereafter	Removes the single-node data-loss mode; DMS gives a rehearsable, resumable cutover	H	included in row 4

OFF-LIST? items

Need	Natural choice	Status	In-list alternative used instead
Shared HTTP session store so any task can serve any request	Amazon ElastiCache for Redis (with Spring Session)	OFF-LIST? — not on the ASSUMED list	ALB application-based sticky sessions now (row 15); move to stateless authentication so no store is needed. Raised as Q13
Cross-region traffic steering with sub-minute failover	AWS Global Accelerator	OFF-LIST?	Route 53 failover records with health checks; slower failover, bounded by DNS TTL. Raised as Q14
Managed CI runners	AWS CodeBuild / CodePipeline	OFF-LIST? (the org's approved pipeline tooling is assumed to be non-AWS)	The organisation's approved pipeline tooling, per G6. Raised as Q17
Centralised WAF/log analytics beyond CloudWatch	Amazon OpenSearch Service	OFF-LIST?	CloudWatch Logs Insights plus forwarding to the central observability platform named in G10. Raised as Q27

Need	Natural choice	Status	In-list alternative used instead
Kubernetes, if the customer's standard container platform is EKS rather than ECS	Amazon EKS	OFF-LIST? on the ASSUMED list	ECS Fargate (row 1). If Q3 confirms EKS is the standard, rows 1, 11 and 13 change; nothing else in this design does. Raised as Q3

4. Guardrail compliance

#	Guardrail	How the target design satisfies it	Exception / open item
G1	Approved-service allowlist	Every service in §3 comes from the ASSUMED list in §1; the five services that did not fit are tagged OFF-LIST? with an in-list substitute rather than silently adopted	The list itself is assumed — Q1
G2	Containers first	ECS Fargate (row 1); no EC2 instances, no VM lift-and-shift; the application is already containerised (Dockerfile:1-37)	Q3 if the standard platform is EKS
G3	Standard target RDBMS	MySQL 8 → Aurora PostgreSQL (rows 4–5); feasible at low code risk because the repo contains zero native SQL, zero stored procedures and zero @Query annotations	None. Data-migration risk remains H
G4	Account vending and promotion	Control Tower dev / QA / prod accounts (row 19); WS7 delivers dev, WS8 QA, WS9/WS10 prod, in that order and no earlier	Q5 (is there an existing landing zone?)
G5	Everything as code	Terraform modules (row 13) replace the raw manifests and the Helm chart; no console changes	None
G6	Approved delivery toolchain	Jenkins CI/CD retired in WS2 (row 12); scans become blocking gates instead of the current advisory ones (vars/sonarqube_code_quality.groovy:3 , vars/trivy_scan.groovy:2)	Q17 names the tool
G7	Production entry criteria	Multi-region prod (Aurora Global Database + standby ECS service + Route 53 failover), ALB load balancing, and a documented DR failover test as a WS9 exit criterion and WS10 entry criterion	Q9 (RTO/RPO define whether warm standby is sufficient), Q14
G8	Security baseline	Tasks and Aurora in private subnets, only the ALB public; TLS at the ALB via ACM and TLS-enforced JDBC to Aurora (replacing useSSL=false , application.properties:3); KMS at rest; IAM task roles instead of the static keys in README.md:36 ; Secrets Manager (row 9)	The committed credentials must be rotated and revoked — WS1, tracked from S1
G9	Approved artefact sources	ECR for images (row 3), approved base image (row 2), vendored front-end assets (row 17), dependencies from the internal Maven mirror	Q16 names the internal mirror
G10	Tagging and observability	Mandatory tag set applied by Terraform default tags; CloudWatch logs/metrics and X-Ray (row 16); alarms on ALB 5xx, task health, Aurora replica lag	Q27 confirms the mandatory tag keys and the central platform

Open Questions

Every question below is a fact the repository cannot settle. Each names the role that owns the answer and the design decision it unblocks. IDs are referenced from `target-state.md` and `migration-plan.md`.

Programme

ID	Question	Owner	Blocks
Q1	No approved ("blessed") AWS service list was supplied. The list in <code>target-state.md §1</code> is an assumption made by us so design could start. Is that list correct, and if not, what is the authoritative list? Everything in the target design is contingent on the answer.	Platform Engineering	WS0 entry; all of <code>target-state.md §3</code>
Q2	No platform migration guardrails were supplied. <code>target-state.md §2</code> uses a default set, labelled as assumed. Do you have your own guardrails, and do they replace or extend these ten?	Platform Engineering	WS0 entry; the guardrail-compliance table
Q3	Is the organisation's standard managed container platform ECS Fargate or EKS? The design assumes ECS; if it is EKS, rows 1, 11 and 13 of the target table change (and much of the existing Kubernetes material becomes reusable).	Platform Engineering	WS4
Q4	Is this repository actually what runs in production? Both Jenkins jobs check out <code>LondheShubham153/Springboot-BankApp (Jenkinsfile:26 , GitOps/Jenkinsfile:21)</code> , and the Kubernetes Deployment runs <code>trainwithshubham/bankapp-eks:v2 (kubernetes/bankapp-deployment.yml:20)</code> , which this pipeline never produces. We may be migrating an artefact nobody deploys.	Application Owner	Everything. This is the first question to answer
Q5	Is there an existing AWS landing zone / Control Tower estate, or is WS0 building one from nothing?	Platform Engineering	WS0 sizing
Q6	Who is the named application owner, cost centre and data classification for the mandatory tag set?	Application Owner	G10, WS5
Q7	What is the target production domain name and which zone hosts it? The repo uses <code>megaproject.trainwithshubham.com (kubernetes/bankapp-ingress.yml:15)</code> , a third-party demo domain.	Application Owner + Platform Engineering	WS9

Database

ID	Question	Owner	Blocks
Q8	What is the production data volume (row counts for <code>account</code> and <code>transaction</code> , total size on disk) and peak transaction rate? The repo shows the schema shape only — two Hibernate-generated tables (<code>model/Account.java</code> , <code>model/Transaction.java</code>) — with no volumetrics.	DBA	WS3 approach (DMS full-load vs full-load + CDC) and cutover window
Q9	What are the RTO and RPO for this workload? Aurora Global Database, backup retention and whether a warm standby is sufficient all follow from the numbers.	Application Owner + DBA	WS9, G7
Q10	The schema is currently generated at runtime by <code>spring.jpa.hibernate.ddl-auto=update (application.properties:9)</code> . Can we take a definitive DDL snapshot of the live production schema? Hibernate's <code>update</code> mode never drops anything, so the live schema may contain columns and indexes no longer present in the entity classes.	DBA	WS3 baseline migration script
Q11	Is there any consumer of the MySQL database other than this application (reporting, ETL, direct queries)? Nothing in the repo indicates one, but the repo cannot prove a negative about external consumers.	DBA + Application Owner	WS3, WS10 cutover scope
Q12	Are <code>BigDecimal</code> balances currently stored as <code>DECIMAL</code> in MySQL, and with what precision/scale? Hibernate's defaults for an unannotated <code>BigDecimal (model/Account.java:19)</code> can differ from what production actually holds; a precision change during the PostgreSQL migration would silently alter monetary values.	DBA	WS3 validation criteria

Platform

ID	Question	Owner	Blocks
Q13	Sessions are held in each JVM's memory and the Ingress sets no session affinity (<code>kubernetes/bankapp-ingress.yml:6-10</code>). Is ALB application-based stickiness acceptable as the interim, or must we move to stateless authentication before production? An in-list session store does not exist on the assumed service list.	Platform Engineering + Application Owner	WS4
Q14	Is Route 53 failover (DNS-TTL-bounded) acceptable for cross-region failover, or is a sub-minute failover requirement going to force Global Accelerator onto the approved list?	Platform Engineering	WS9, G7
Q15	Which approved base container image should replace <code>openjdk:17-alpine (Dockerfile:28)</code> , and where is it published?	Platform Engineering	WS2
Q16	Which internal Maven repository must the build resolve dependencies from? The build currently resolves from Maven Central via the wrapper (<code>.mvn/wrapper/maven-wrapper.properties</code>).	Platform Engineering	WS2, G9

Delivery

ID	Question	Owner	Blocks
Q17	Which pipeline product is the approved delivery toolchain, and who owns onboarding this repository onto it?	Release Manager	WS2
Q18	Is the Jenkins estate (Jenkinsfile , GitOps/Jenkinsfile , the Shared library in vars/) shared with other applications, or can it be decommissioned with this workload?	Release Manager	WS2 exit
Q19	ArgoCD is documented (kubernetes/README.md:116-133) but no Application manifest is committed. Is ArgoCD actually in use for this workload today? If so, its configuration lives outside this repository and we need it.	Release Manager + Platform Engineering	WS2 current-state completeness
Q20	What release cadence and change-approval process applies? The CD job commits directly to a branch with no review (GitOps/Jenkinsfile:50-63).	Release Manager	WS2, WS10

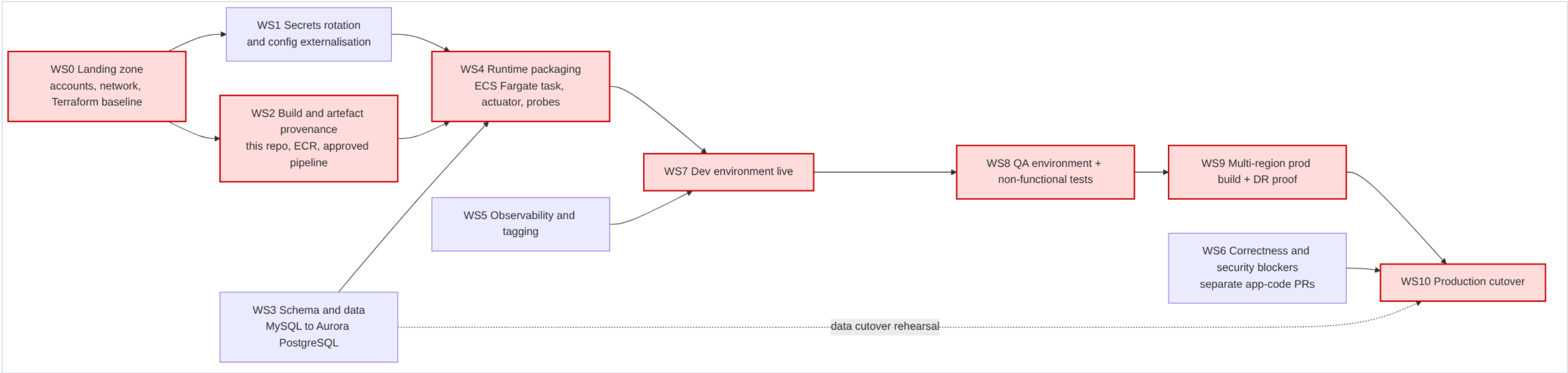
Security

ID	Question	Owner	Blocks
Q21	Database credentials are committed in four locations (current-state.md §7 S1). Have these values ever been used against a production database, and if so, when will they be rotated and revoked? Values are deliberately not reproduced here or anywhere in this package.	Security	WS1; WS10 entry criterion
Q22	CSRF is disabled (config/SecurityConfig.java:30) on a cookie-session application with state-changing POSTs. Is there a compensating control today (WAF rule, gateway), or is this an unmitigated exposure that must be fixed before cutover?	Security	WS6
Q23	Is there a required authorisation/segregation-of-duties model? Today every user receives the single authority "USER" (service/AccountService.java:99-101).	Security + Application Owner	WS6 scope
Q24	What are the regulatory audit-logging requirements for financial events? The application logs no financial events today (current-state.md §7 S7).	Security + Application Owner	WS5, WS6
Q25	Is TLS to the database mandatory in all environments? Every environment definition currently sets useSSL=false (application.properties:3 , kubernetes/configmap.yaml:8 , docker-compose.yml:25).	Security	WS4, G8
Q26	Which internal registry and scanning policy governs container images, and what severity threshold blocks a release? Today Trivy runs with no threshold (vars/trivy_scan.groovy:2) and the SonarQube gate is explicitly non-blocking (vars/sonarqube_code_quality.groovy:3).	Security + Release Manager	WS2

Operations

ID	Question	Owner	Blocks
Q27	Which central observability platform must receive logs and metrics, and what are the mandatory tag keys?	Platform Engineering	WS5, G10
Q28	What is the acceptable cutover window (planned downtime) for the database switch, and who signs off on it?	Application Owner + Release Manager	WS10
Q29	Who operates this workload after cutover, and what on-call/runbook standard must be met before production entry?	Application Owner + Platform Engineering	WS10
Q30	What functional acceptance criteria will be used to validate the migrated system? Automated coverage today is a single contextLoads() test (src/test/java/com/example/bankapp/BankappApplicationTests.java:9-11), so acceptance will be manual unless a test suite is built (WS6).	Application Owner + QA	WS8, WS10

Migration Plan



Effort is expressed in Devin sessions for one engineer (one session ≈ one to two human-weeks of equivalent work). Sequencing, not duration, is what matters here: the calendar is dominated by answers to the open questions and by account vending, not by engineering throughput.

Critical path: WS0 → WS2 → WS4 → WS7 → WS8 → WS9 → WS10. WS3 (data) runs in parallel from the start but joins the path at WS4 and must be rehearsed again inside WS10. WS6 is off the build path but is a hard gate on WS10.

Programme-level entry criteria (before WS0 starts): Q1 (approved service list) and Q2 (guardrails) answered, and Q4 answered — if the deployed artefact is built from `LondheShubham153/Springboot-BankApp` rather than this repository (`Jenkinsfile:26` , `Git0ps/Jenkinsfile:21`), the migration scope changes before anything else does.

WS0 — Landing zone and account vending

Entry criteria	Q1, Q2, Q5 answered; Q6 tag values supplied
Dependencies	None
Effort	1 session (engineering) plus account-vending lead time

- Vend dev, QA and prod accounts through Control Tower (G4). QA is not requested until dev is running.
- Terraform baseline: remote state, provider/version pinning, module layout, mandatory default tags (G5, G10).
- VPC per account: three AZs, private subnets for tasks and Aurora, public subnets for the ALB only (G8).
- Baseline IAM roles and the SCP posture that enforces the approved-service allowlist (G1).

Exit: `terraform apply` from a pipeline creates an empty, compliant dev VPC. No console-built resource exists.

WS1 — Secrets rotation and config externalisation

Entry criteria	WS0 dev account exists; Q21 answered (were the committed credentials ever live?)
Dependencies	WS0
Effort	1 session

- **Rotate and revoke every credential committed to this repository.** Four locations are recorded as S1 in `current-state.md` §7 (`application.properties:4-5` , `docker-compose.yml:7` , `docker-compose.yml:26` , `kubernetes/secrets.yaml:8-9` , `helm/bankapp/values.yaml:52-53`). Values are not reproduced in this package. Base64 in the Kubernetes Secret is encoding, not protection.
- Treat the values as public: rotate at the database, then revoke, then confirm no other consumer broke (Q11).
- Create the Secrets Manager secret and SSM parameters that WS4 will consume; enable rotation (G8).

- Purging the values from Git history is a separate decision for Security — rotation is what actually closes the exposure.

Exit: Security confirms no credential that appears in this repository is valid against any live database.

WS2 — Build and artefact provenance

Entry criteria	Q4, Q15, Q16, Q17, Q26 answered
Dependencies	WS0
Effort	2 sessions

- Point the pipeline at **this** repository. Today both Jenkins jobs build a different one (C1 in `current-state.md §6`) — no change committed here is ever built.
- Onboard to the approved CD tooling (G6); retire `Jenkinsfile`, `GitOps/Jenkinsfile` and the `Shared` library in `vars/` once Q18 confirms nothing else uses them.
- Rebuild the image on the approved base (Q15) with a non-root `USER` (`Dockerfile:21-37` declares none) and stop skipping tests in the image build (`Dockerfile:25`).
- Push to ECR with immutable tags and scan-on-push (G9); one image identity replaces the four in C3.
- Make the scans blocking: Trivy currently runs with no severity threshold (`vars/trivy_scan.groovy:2`) and the SonarQube quality gate is explicitly non-blocking (`vars/sonarqube_code_quality.groovy:3`).
- Resolve dependencies from the internal Maven mirror (Q16).

Exit: A commit to this repository produces a scanned, immutably tagged image in ECR, and a failing scan fails the build.

WS3 — Schema and data migration (MySQL → Aurora PostgreSQL)

Entry criteria	Q8 (volumes), Q10 (live DDL snapshot), Q11 (other consumers), Q12 (decimal precision) answered
Dependencies	WS0 for the target Aurora cluster
Effort	3 sessions

This is a workstream, not a table row, but the code makes it far cheaper than a typical engine change: the repository contains **zero** native SQL, `@Query` annotations, `JdbcTemplate`, `EntityManager` and stored-procedure usage (zero-counts recorded in `current-state.md §2`). All access is Spring Data JPA through two repository interfaces. The risk is in the data and the runtime schema, not the queries.

1. Capture the authoritative production DDL (Q10). `spring.jpa.hibernate.ddl-auto=update` (`application.properties:9`) never drops anything, so the live schema is a superset of the entities.
2. Author the PostgreSQL baseline as versioned migrations (Flyway/Liquibase) and switch the application to `ddl-auto=validate` with `PostgreSQLDialect` (replacing `MySQL8Dialect`, `application.properties:10`).
3. Convert types deliberately: `BigDecimal` balances (`model/Account.java:19`) and amounts (`model/Transaction.java:13`) must land as `NUMERIC` with the precision confirmed in Q12 — a silent precision change alters money.
4. DMS full load, then CDC if Q8’s volumes make a full-load-only window unacceptable.
5. Row-count and checksum validation per table; reconcile every account balance against the sum of its transactions before and after.

Exit: A rehearsed migration run with signed-off validation output, and a documented rollback.

WS4 — Runtime packaging on ECS Fargate

Entry criteria	WS1 secrets exist, WS2 produces an ECR image, WS3 has a target schema; Q3, Q13, Q25 answered
Dependencies	WS1, WS2, WS3
Effort	2 sessions

- Terraform: ECS cluster, task definition, service (min 3 tasks / 3 AZs), ALB, target group, ACM certificate, WAF.
- Task role reads the Secrets Manager secret by ARN; no static credentials in the task definition (G8).
- JDBC to Aurora over TLS — every current environment sets `useSSL=false` (`application.properties:3`, `kubernetes/configmap.yaml:8`, `docker-compose.yml:25`), which does not survive the security baseline (Q25).
- Add `spring-boot-starter-actuator` (absent from `pom.xml`) so `/actuator/health` — which the Helm chart and Compose file already probe (C4) — actually exists, and wire it to the ALB health check.
- Session handling: ALB application-cookie stickiness as the interim (Q13). Today sessions live in each JVM with no affinity configured across two replicas.
- Service Auto Scaling replaces the HPA, and the conflicting VPA in the Helm chart (C7) is not carried over.

Exit: The application serves traffic through the ALB in dev against Aurora, with a green health check.

WS5 — Observability, tagging and cost attribution

Entry criteria	Q6 and Q27 answered

Dependencies	WS0 (can run in parallel with WS2–WS4)
Effort	1 session

- Structured JSON application logging to CloudWatch Logs; `spring.jpa.show-sql=true` (`application.properties:11`) is turned off outside dev.
- Alarms: ALB 5xx and target health, ECS task restarts, Aurora CPU/connections/replica lag, WAF blocks.
- X-Ray tracing; mandatory tag set applied via Terraform default tags (G10).
- Forward to the central observability platform named in Q27.

Exit: A dashboard and alarm set exist for dev and are reproduced by Terraform in every environment.

WS6 — Correctness and security blockers (application-code work)

Entry criteria	Q22, Q23, Q24, Q30 answered; owner assigned per item
Dependencies	None technically — but WS10 cannot start until this closes
Effort	3 sessions

Nothing in this workstream is in the docs-only PR that carries this plan. Each item is a separate application-code change, tracked from `current-state.md` §7:

Ref	Blocker	Evidence
S2	Transfers perform four writes with no transaction boundary; <code>@Transactional</code> appears zero times in the codebase	<code>service/AccountService.java:103-135</code> , zero <code>@Transactional</code> in <code>src/</code>
S3	No positive-amount validation on deposit, withdraw or transfer — negative amounts are accepted	<code>service/AccountService.java:51</code> , <code>:64</code> , <code>:103</code> , <code>controller/BankController.java:51</code> , <code>:59</code> , <code>:83</code>
S4	CSRF disabled on a cookie-session app with state-changing POSTs	<code>config/SecurityConfig.java:30</code>
S5	Single hardcoded <code>"USER"</code> authority; no maker-checker, no limits	<code>service/AccountService.java:99-101</code>
S6	Hibernate mutates the schema at runtime	<code>application.properties:9</code>
S7	No audit logging of financial events	absence across <code>service/</code> , <code>controller/</code>
S8	Database TLS disabled in every environment	<code>application.properties:3</code> , <code>kubernetes/configmap.yaml:8</code>
S9	Container runs as root	<code>Dockerfile:21-37</code>
S10	Automated test coverage is one <code>contextLoads()</code>	<code>src/test/java/com/example/bankapp/BankappApplicationTests.java:9-11</code>

S10 is the enabler: without a real test suite there is no way to evidence that the PostgreSQL migration preserved behaviour, so build it first (Q30 defines the acceptance criteria).

Exit: Each blocker is fixed and merged, or formally accepted in writing by Security and the Application Owner.

WS7 — Dev environment live

Entry criteria	WS4 exits; WS5 alarms in place
Dependencies	WS4, WS5
Effort	0.5 session

- Multi-AZ, single region (the stated dev posture).
- Full pipeline path exercised: `commit` → `scan` → `ECR` → `ECS` deploy, with no manual step.
- Dev running is the precondition for requesting the QA account under G4.

Exit: Dev is deployed only by the pipeline, and a redeploy from a clean state is demonstrated.

WS8 — QA environment and non-functional validation

Entry criteria	WS7 exits; QA account vended; Q30 acceptance criteria agreed
Dependencies	WS7
Effort	1.5 sessions

- Terraform-identical to dev; environment differences only in variables (G5).
- Functional acceptance against the migrated Postgres schema, with a production-representative data set from the WS3 rehearsal.
- Load test to establish the real scaling profile — the current HPA settings (1–5 tasks at 40% CPU, `kubernetes/bankapp-hpa.yml:11-19`) are inherited assumptions with no measurement behind them.
- Security validation: WAF rules, penetration test, confirmation that no public endpoint exists other than the ALB.

Exit: QA signed off by the Application Owner against the Q30 criteria.

WS9 — Multi-region production build and DR proof

Entry criteria	WS8 signed off; Q7 (domain), Q9 (RTO/RPO), Q14 (failover mechanism) answered
Dependencies	WS8
Effort	2 sessions

- Prod account: primary region full stack, secondary region ALB + standby ECS service + Aurora Global Database secondary.
- Route 53 records in the corporate zone with health checks and failover routing (Q7).
- **Documented DR failover test executed against the production stack before any customer traffic:** promote the secondary Aurora cluster, shift Route 53, verify the application serves, and measure achieved RTO/RPO against Q9. This is guardrail G7 and it is a test, not a design review.

Exit: A signed DR test report with measured RTO/RPO, and the prod stack idle and ready.

WS10 — Production cutover (final workstream)

Dependencies	WS9, WS6, WS3 rehearsal
Effort	1 session plus the cutover window

Entry criteria — all must be true before the window opens

1. **WS6 closed.** Every blocker S2–S10 is fixed and merged, or formally accepted in writing by Security and the Application Owner. Migrating an unfixed unatomic-transfer defect into a multi-region production estate makes it harder to fix, not easier.
2. **WS1 closed.** Security confirms every credential committed to this repository has been rotated and revoked (S1, Q21).
3. **G7 production entry criteria met:** multi-region prod stack live, ALB load balancing in place, and a **completed DR failover test with measured RTO/RPO** accepted against Q9 (WS9 exit).
4. **WS3 rehearsed at production scale** with validation output signed off by the DBA, and a rollback procedure that has itself been tested.
5. **QA sign-off** against the Q30 acceptance criteria (WS8 exit).
6. Cutover window and downtime agreed and approved (Q28).
7. Operating model in place: named owner, on-call rota, runbooks (Q29), and the tag set applied (Q6).
8. Legacy decommissioning plan agreed, including the EKS cluster, the in-cluster MySQL and its hostPath volume (`kubernetes/persistent-volume.yml:14-15`), and the Docker Hub images.

Cutover sequence

1. Freeze writes to the legacy application at the start of the window.
2. Final DMS sync; stop replication; run the WS3 validation suite; DBA signs off on the data.
3. Scale up the ECS service in the primary region; verify health checks and a smoke transaction set.
4. Shift Route 53 to the ALB (Q7); watch WAF, ALB 5xx and Aurora metrics through the WS5 alarms.
5. Hypercare: legacy stack kept intact but read-only for the agreed rollback period.
6. Roll back by reverting the Route 53 record and unfreezing legacy — valid only while legacy holds the authoritative data, which is why the write freeze in step 1 is non-negotiable.
7. After hypercare, decommission per entry criterion 8.

Exit: Production traffic served from AWS, legacy decommissioned, DR test repeatable on a schedule.